

EduCore — LMS & Digital Management System

Project Goal

Create a full-stack Learning Management System (LMS) and Digital Management System (DMS) that simulates a real-world educational platform.

The platform should include:

- Student management
 - Instructor management
 - Course and class management
 - Assignments and grading
 - Attendance tracking
 - Notifications
 - Reports
 - CMS functionality
 - Real-time features
 - SQL + NoSQL integration
-

Recommended Tech Stack

Frontend

- React + TypeScript
- Tailwind CSS
- Shadcn UI
- React Hook Form
- Zod Validation
- TanStack Query
- Socket.IO Client
- Recharts

Backend

- Node.js
- Express.js
- TypeScript
- REST API
- JWT Authentication
- Socket.IO
- Multer/File Uploads

Databases

SQL Database (PostgreSQL or MySQL)

Used for structured and relational data.

MongoDB

Used for flexible and dynamic content.

Additional Tools

- GitHub
 - Docker
 - Swagger
 - Postman
 - ClickUp/Trello/Jira
 - Cloudinary
-

Database Architecture

SQL Database Tables

Users

- id
- fullName
- email
- password
- role
- createdAt

Students

- id
- userId
- studentCode
- department
- semester

Instructors

- id
- userId
- specialization

Courses

- id
- title
- description
- instructorId

Classes

- id
- courseId
- room
- schedule

Enrollments

- id
- studentId
- classId

Attendance

- id
- studentId
- classId
- status
- date

Assignments

- id
- courseId
- title
- dueDate

Submissions

- id
 - assignmentId
 - studentId
 - fileUrl
 - grade
-

MongoDB Collections

Lessons

Stores dynamic lesson content.

Notifications

Stores real-time notifications.

Activity Logs

Stores user activities.

Announcements

Stores system/course announcements.

CMS Content

Stores editable page content.

Quiz Questions

Stores flexible quiz structures.

Main System Roles

Admin

Can:

- Manage users
- Manage instructors
- Manage students
- Manage classes
- Manage reports
- View analytics
- Configure CMS

Instructor

Can:

- Create courses
- Upload lessons
- Create assignments
- Grade submissions
- Track attendance
- Send notifications

Student

Can:

- View lessons
 - Submit assignments
 - Track grades
 - View notifications
 - View progress reports
-

Features Required For Maximum Grade

Mandatory Features

1. Frontend Framework

Use React + TypeScript.

2. Backend Framework

Use Node.js + Express.

3. SQL Database

Use PostgreSQL or MySQL.

4. NoSQL Database

Use MongoDB.

5. Git Version Control

Maintain:

- clean commits
- proper branches
- organized repository

6. Project Management Tool

Use:

- ClickUp
 - Trello
 - Jira
-

Advanced Features

1. Authentication & Authorization

Implement:

- JWT Access Token
- Refresh Token
- Role-Based Access Control
- Protected Routes

2. Real-Time Notifications

Using Socket.IO:

- Assignment alerts
- Grade updates
- New announcements

3. Advanced Search

Search:

- students
- instructors
- courses
- lessons
- assignments

4. Dynamic Reports

Generate:

- attendance reports
- student performance reports
- class analytics

5. Import & Export

Support:

- CSV
- Excel
- JSON

6. CMS System

Allow admins/instructors to:

- create lessons
- edit content
- upload images
- manage page blocks

7. Docker

Dockerize:

- frontend
- backend
- SQL database
- MongoDB

8. AI Recommendation System

Recommend:

- lessons
 - assignments
 - study material based on weak performance.
-

Recommended Development Order

Phase 1 — Planning

Tasks

- Define project scope
 - Create ERD diagram
 - Design MongoDB collections
 - Create use case diagrams
 - Create GitHub repository
 - Setup ClickUp/Trello board
 - Define team roles
-

Phase 2 — Backend Setup

Tasks

- Setup Express server
 - Configure TypeScript
 - Configure PostgreSQL/MySQL
 - Configure MongoDB
 - Create folder structure
 - Create middleware
 - Setup environment variables
 - Setup error handling
-

Phase 3 — Authentication System

Tasks

- Register API
 - Login API
 - Refresh token system
 - Password hashing
 - JWT middleware
 - Role middleware
 - Protected frontend routes
-

Phase 4 — DMS Development

Tasks

- Student management
 - Instructor management
 - Class management
 - Schedule management
 - Attendance management
 - Dashboard analytics
-

Phase 5 — LMS Development

Tasks

- Course creation
 - Module creation
 - Lesson CMS
 - Assignment management
 - Submission system
 - Grading system
 - Student progress tracking
-

Phase 6 — Advanced Features

Tasks

- Real-time notifications
 - Advanced search
 - Report generation
 - CSV export/import
 - CMS customization
 - AI recommendation system
-

Phase 7 — Finalization

Tasks

- Responsive design improvements

- Testing
 - Bug fixing
 - Swagger documentation
 - Seed database data
 - Final presentation
 - Final report
-

Recommended Folder Structure

```
educore/  
  frontend/  
    src/  
      components/  
      pages/  
      hooks/  
      services/  
      features/  
      utils/  
      types/  
  
  backend/  
    src/  
      modules/  
        auth/  
        users/  
        students/  
        instructors/  
        courses/  
        assignments/  
        attendance/  
        notifications/  
        reports/  
  
      middleware/  
      config/  
      database/  
      utils/  
  
  docs/  
    erd.png  
    api-docs.md  
    project-plan.md
```

Suggested Git Branch Structure

Main Branches

- main
- development

Feature Branches

- feature/auth
 - feature/dashboard
 - feature/courses
 - feature/assignments
 - feature/notifications
 - feature/reports
-

API Recommendations

REST Endpoints

Auth

- POST /auth/register
- POST /auth/login
- POST /auth/refresh

Students

- GET /students
- POST /students
- PUT /students/:id
- DELETE /students/:id

Courses

- GET /courses
- POST /courses
- PUT /courses/:id

Assignments

- POST /assignments
- GET /assignments
- POST /submissions

Dashboard Ideas

Admin Dashboard

- Total students
- Total instructors
- Attendance statistics
- Recent activity
- Course analytics

Instructor Dashboard

- Course overview
- Student progress
- Assignment statistics
- Notifications

Student Dashboard

- Current courses
- Grades
- Attendance
- Upcoming assignments

ClickUp / Trello Sprint Plan

Sprint 1 — Setup

- Create GitHub repository
- Setup frontend
- Setup backend
- Configure databases
- Configure Docker
- Setup environment variables

Sprint 2 — Authentication

- Create auth APIs
- Create role system
- Create protected routes
- Create login/register pages

Sprint 3 — DMS

- Student management
- Instructor management
- Attendance system
- Schedule system

Sprint 4 — LMS

- Courses
- Lessons
- Assignments
- Submissions
- Grades

Sprint 5 — Advanced Features

- Notifications
- Reports
- Search
- CSV import/export
- CMS
- AI recommendations

Sprint 6 — Finalization

- Testing
- Responsive fixes
- Documentation
- Final presentation
- Bug fixing

Final Recommendation

To achieve the highest grade:

Focus on:

- clean architecture
- strong authentication
- professional UI/UX
- real-time functionality

- proper database design
- advanced reports
- organized Git workflow
- clean documentation

Avoid overcomplicating the project with unnecessary features.

A polished, stable, well-documented system with advanced core features will score higher than a very large unfinished system.